

数据挖掘论文阅读：PINN

- [@概要](#)
- [@一、引言](#)
- [@二、PINN的组成部分](#)
- [@三、用PINN解偏微分方程](#)
- [@四、PINN的数据、应用和软件](#)
- [@五、PINN未来的挑战与方向](#)
- [@六、读后总结](#)
- [@AI使用情况](#)

说明：文字黑体表示源自原文，加粗的是我觉得**重要的关键词**，**青色的是我自己思考的评注**

概要

1. 背景：PINN是一类编码了数学方程（例如偏微分方程）的神经网络，PINN已经被用于解决多种方程（偏微分方程，差分方程等），在多任务学习方面比神经网络做得更好。
2. 文章内容：叙述各种PINN模型的特征和优缺点，并且介绍其他基于配置的内嵌物理信息的模型
3. 文章发现：现有研究大多通过不同的激活函数、梯度优化技术、神经网络结构和损失函数结构来定制PINN
4. 意义：发掘在什么情况下PINN比传统数值方法（例如有限元方法）更优，特别是一些悬而未决的理论问题。

一、引言

一、一些PDE可能有显著的非线性，对流占优或者激波等问题，用传统的数值方法难以求解。深度学习因其泛化能力已经成为了科学计算的新范式，可以是动态系统预测的元模型。神经网络代表了复杂系统中潜在的非线性输入输出关系，不能避免维数灾难，但是仍然是用于解偏微分方程的有效方法。

二、用机器学习方法解PDE会是一个长期的研究热点。现代方法基于神经网络技术，得益于现代优化方法框架和自动微分的发展。

自动微分（来自Qwen3-Max）：

自动微分的核心思想

任何计算机程序计算的函数，都可以看作一系列**基本运算**（如 $+$, $-$, $\times$, $\div$, $\sin$, $\exp$, $\log$ 等）的组合。

自动微分利用**链式法则**（Chain Rule），将这些基本运算的导数组合起来，从而得到整体函数的导数。

两种主要模式

1. 前向模式（Forward Mode）

- 沿计算方向同时计算函数值和导数。
- 适合输入少、输出多的情况（如 $f: \mathbb{R} \rightarrow \mathbb{R}^n$ ）。
- 一次前向传播可得一个方向的导数。

2. 反向模式（Reverse Mode）

- 先正向计算函数值，再反向传播导数（类似反向传播算法）。
- 适合输入多、输出少的情况（如 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ ——这正是深度学习中损失函数的情形！）
- 一次反向传播可得所有输入变量的梯度。

三、Kim等人发展了对于这类称为整合信息模型模型的分类方法，由三种概念组织：1、深度神经网络种类；2、物理信息的呈现方式；3、物理信息整合进入模型的方式。

四、文章揭示PINNs怎么利用神经网络的特性，其中的物理信息是怎么来的，以及文献中已经解决了哪些物理问题。

我的问题：PINN是Physics-Informed Neural Network，物理信息是怎么影响（informed）神经网络的？给神经网络带来了怎样的变化？

1.1 PINNs是什么

PINNs的目的：解偏微分方程。

训练方法：建立神经网络，最小化损失函数，损失函数应含反映时空域边界上初始条件和边界条件的项，以及域中选定点（称为配置点）上的偏微分方程残差。

训练后的深度神经网络可以：输入一个积分域中的点，输出一个PDE在该点的估计解

基本思想：将支配物理的方程编码到残差网络里，一种不需要标注数据的无监督学习方法，将数学问题从解方程变为最小化含残差项的损失函数

PINNs并不是简单的通过状态-值拟合出方程解，而是考虑到了隐含的物理情景（PDE）。

PINN有非常多的变体，并且延申到了解积分微分方程和随机微分方程等领域

相较于传统方法：1、PINN是无网格的，可以在训练阶段后按需计算解决方案；2、使用解析梯度，使得方程解可微；3、通过同一个优化问题方便同时解决[前向和倒向问题](#)，因为两个问题使用的代码几乎是一样的。

我的问题：具体的一个PINN解决的问题长什么样，PINN是如何解决它的？损失函数的具体形式是怎样的（特别是如果有例子的话），可以用什么方法优化？

1.2 本文的主要内容

文章将介绍：如何使用PINNs解决困难的科学计算问题、**PINNs的组成部分**（最感兴趣）、和学习理论的概念联系、已经成熟的工具箱、PINNs未来的研究方向和最近的研究热点、准确性和收敛问题以及各种文献中如何通过配置深度、层大小、激活函数和使用迁移学习来针对不同问题定制PINN求解器。

还有最基本的问题：什么是PINN，其优势和缺点是什么

二、PINN的组成部分

一、PINN解决微分方程的类型：

$$\begin{aligned}\mathcal{F}(u(z); \gamma) &= f(z), \quad z \in \Omega, \\ \mathcal{B}(u(z)) &= g(z), \quad z \in \partial\Omega,\end{aligned}\tag{1}$$

定义在区域 $\Omega \subset \mathbb{R}^d$ 上，其边界为 $\partial\Omega$ 。其中 $z := [x_1, \dots, x_{d-1}; t]$ 表示时空坐标向量， u 是待求解的未知函数， γ 是与物理问题相关的参数， f 是描述问题数据的函数， $\mathcal{F}$ 是一个非线性微分算子。

$\mathcal{B}$ 表示与该问题相关的任意初始条件或边界条件的算子， g 表示对应的**边界函数**，这些边界条件可以是狄利克雷（Dirichlet）、诺依曼（Neumann）、罗宾（Robin）或周期性边界条件（periodic boundary conditions）。

边界条件（来自Qwen3-Max）：

在时空统一的框架下，初始条件可以被看作是一种特殊的边界条件（Dirichlet 类型）

- 边界条件类型：

- Dirichlet: 直接指定解在边界上的值，如 $u = g$ 。
- Dirichlet: 直接指定解在边界上的值，如 $u = g$ 。
- Neumann: 指定解的法向导数，如 $\frac{\partial u}{\partial n} = g$ 。
- Robin: 两者的线性组合，如 $au + b\frac{\partial u}{\partial n} = g$ 。
- Periodic: 解在边界两侧周期性重复，常用于模拟无限或循环系统。

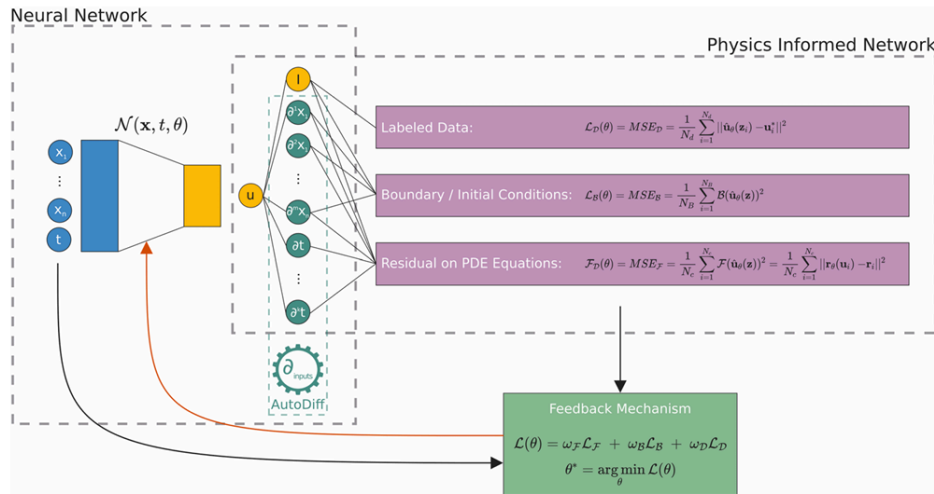
这类方程描述了大量实际的物理问题。在正向问题中, γ 是给定的，需要找到 z 对应的 u ；在倒向问题中 γ 是待定的，需要通过数据确定。

符号设定： θ 指代神经网络的待定参数， γ 指代反问题中的待定参数

二、 θ 应该从微分方程，边界条件和已知数据估计，这三项需要分配权重：

$$\theta^* = \arg \min_{\theta} (\omega_{\mathcal{F}} \mathcal{L}_{\mathcal{F}}(\theta) + \omega_{\mathcal{B}} \mathcal{L}_{\mathcal{B}}(\theta) + \omega_d \mathcal{L}_{\text{data}}(\theta))$$

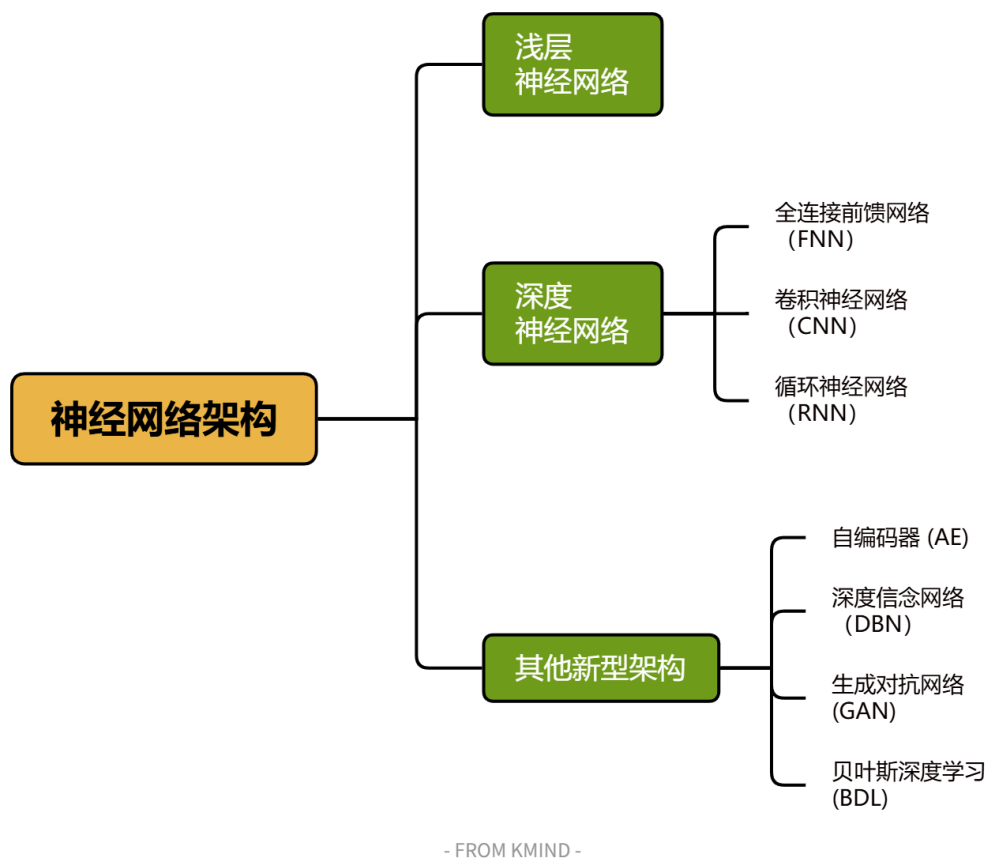
在**前向问题**中，PINN的训练不需要真实解作为有标签的数据，可以视为一种**无监督学习**的算法；在**反向问题**中，需要有 $\mathcal{L}_{\text{data}}(\theta)$ ，作为数据拟合项，被视为一种有监督学习算法。



2.1 神经网络架构

一、万能逼近定理：任何连续函数都可以通过只有一个隐藏层和有限数量神经元的多层感知器任意逼近。这表明神经网络可以简洁地表示复杂函数，但是确定神经网络的参数和具体的构造却很

困难。主要的构造：只有一层隐藏层的浅层神经网络和使用两层以上的深度神经网络，而怎么确定深度神经网络的结构也是研究的一个要点。



二、 L 层深度神经网络的表示：

$$u_{\theta}(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x). \quad (3)$$

其中： $u_{\theta}(x)$ ：表示由神经网络近似的函数，输入为 x ，参数为 θ ； f_i ：第 i 层的映射函数，定义在两个内积空间 E_i 和 H_i 上，即 $f_i : E_i \times H_i \rightarrow \mathbb{R}$ ； $\circ$ ：表示函数复合运算，即 $f_2 \circ f_1(x) = f_2(f_1(x))$ ； x ：输入变量，也称为状态变量 (state variables)； θ_i ：第 i 层的参数集合，所有层的参数共同构成整体参数集 θ 。

2.1.1 前馈神经网络

前馈神经网络也叫**多层感知机 (MLP)**，是只向前传播的DNN。在全连接的NN中，相邻层的神经元相互连接，同层神经元不连接，神经元中是一个将权值和函数与偏置因子输入到激活函数的数学算子。

在有一层输入层，一层输出层和 $K - 2$ 层隐藏层的MLP中，(3)式中的各项可以表述为

$$f_i(x_i; W_i, b_i) = \alpha_i(W_i \cdot x_i + b_i)$$

$$u_\theta(x) = C_K \circ \alpha \circ C_{K-1} \circ \cdots \circ \alpha \circ C_1(x), \quad (4)$$

其中，对于任意 $1 \leq k \leq K$ ，定义：

$$C_k(x_k) = W_k x_k + b_k. \quad (5)$$

一、全连接前馈神经网络

基础的神经网络模型（**上课学的那种**）。

研究趋势：改变FFNN的神经元层数、单层神经元的数量、神经元内置的激活函数（文献主要提到了双曲正切函数、反正切函数）、所解的微分方程的类型，以及改变这些条件对估计准确性的影响

就微分方程的类型而言，文章提到了Kharazmi等人的hp-VPINN，体现了微分问题会影响网格架构，采用四层架构，每层二十个神经元，但对于具有精确解的双间断性的对流方程，他们使用了一个八层的深度网络。对约束问题（不将初值等约束条件作为损失函数的优化目标，而是强制其满足约束），Zhu等人使用了五层每层250个神经元的网络。**这表明方程和约束越复杂，所需的网络规模可能就越大。**

二、混合全连接网络 (Multiple FFNN)

用多个FFNN解决问题的方案。

将一个问题拆分成多个问题让DNN解决：Cai等人在两相Stefan问题中，将不同材料的未知界面和这两个相的温度分布分别用一个DNN模拟。Moseley等人将定义域拆分为多个子域，在各个子域上分别用一个NN。Lu等人将输入空间和输出函数的域分别用一个DNN编码，称为分支网和主干网，这类方法被称为DeepONet，因为对两个网络的拓扑特性没有要求这种方法较为通用，两个子网络已经被Lin等人实现为FFNNs

三、浅层神经网络

用层数较少的MLP解决问题的方案。例如稀疏神经网络（Sparse NN，权重矩阵或连接结构大部分元素为零的矩阵）和极限学习机（只有一层隐藏层，输入层到隐藏层）

来自Qwen3-Max：

稀疏神经网络是指权重矩阵或连接结构中大部分元素为零（即“稀疏”）的神经网络。这种稀疏性可以出现在：

- **权重稀疏**：训练后剪枝，使许多权重为 0；
- **结构稀疏**：预先设计连接模式（如只连接局部区域），如卷积网络本身具有稀疏连接特性。

ELM 是一种单隐层前馈神经网络（SLFN），其特点是：

- 输入层到隐藏层的权重和偏置随机初始化且固定不变；
- 仅输出层权重通过解析方法（如最小二乘）一次性求解，无需迭代训练。

使用浅层而不是深层的动机：层数是对非线性关系的建模，在真实（工程）问题中，深层的网络会导致高训练成本和计算效率低的问题。

Schiassi等人：ELM→X-TFC；PINN+ELM→PIELM；Ramabathiran和Ramachandran提出了一种稀疏的、基于物理的、部分可解释的神经网络（SPINN），该架构是稀疏的，使用内核网络，产生可解释结果，所需参数比完全连接的解决方案少，他们考虑各种核，如径向基函数（RBF）、软核或高斯核，并将其概念证明架构应用于各种数学问题

四、激活函数

常见激活函数：ReLU, Sigmoid, Tanh（双曲正切）

推荐使用可变的激活函数。Swish: $x \cdot \text{Sigmoid}(\beta x)$ ，其中 β 是一个可学习的参数， $\text{Sigmoid}(\cdot)$ 是可选取的S型曲线，常见的有：Logistic Sigmoid函数、双曲正切函数（由于在无穷可微，更常用）、ReLU和Leaky ReLU（在正数域上和ReLU一样，在负数域上定义为 αx ， α 是一个极小的正数）。

Cheng and Zhang使用残差网络（不直接学习映射，而通过学习残差映射再回到原映射）来提升全连接神经网络的稳定性，他们也证明了Swish形式的激活函数在提升NN的一致性和准确性方面比其他形式更好

由于和二阶微分求和有关，选择一个合适的激活函数在PINN中非常重要。例如，将PDE重新缩放为无量纲形式时，应选择 $[0, 1]$ （ $[-1, 1]$ 也行），因为大多数激活函数在0附近才是非线性的，才能拟合非线性关系。

平滑的激活函数可以保证PINN的正则性，从而使PINNs的泛化误差（在未见数据上的预测误差）的估计能够保持真实

2.1.2 卷积神经网络

卷积神经网络（CNN）在进入网络之前通过交替的卷积层和池化层降低数据的维度。其中卷积核（一个阶数较小的可学习的权重矩阵）会在输入数据上滑动进行卷积操作；池化层则进行子采样，例如图像数据中一个像素及其周围的像素用9个像素中的最大值（最大池化）/平均值（平均池化）代表，从而降低数据维度。

逐层函数 f 的具体表达为：

$$f_i(x_i; W_i) = \Phi_i(\alpha_i(C_i(W_i, x_i)))$$

其中： x_i 为第 i 层的输入（通常是前一层的输出特征图）； W_i 是**卷积核权重**，第 i 层的可学习参数； $C_i(W_i, x_i)$ 表示用卷积核 W_i 对输入 x_i 进行卷积运算； α_i 为逐元素非线性激活函数（如 ReLU、Sigmoid 等），用于引入非线性特征； Φ_i 是最大池化映射（max-pooling map），用于对激活后的特征图进行下采样，降低空间维度。

卷积操作具有平移不变性，在图像数据中这意味着边缘、纹理等特征和其出现的位置关系不大，仍然会在卷积层中得到表示，所以特别适合用来处理图像数据，在和图有关的领域CNN已经被用于PINN了。

一、卷积神经网络架构

CNN是为处理图像数据而生的，更适合处理类图像数据而并不适合直接解决非常规和非统一的网格几何结构的科学计算问题，基于欧式距离的卷积核在非统一的网格中会失去平移不变性。

Gao等人则开发了具有几何适应性的物理信息卷积神经网络（PhyGeoNet），通过坐标变换将解域从不规则物理域转化为标准的矩形域。同时，边界条件是强约束的，所以它也是一个 physics constrained neural network（物理约束神经网络，PCNN）。

Fang发现拉普拉斯算子可以通过有限差分标准核被离散化为卷积算子，于是他通过有限体积数值方法将CNN结构用于提升PINN性能上：在流形而不是笛卡尔网格中计算卷积，用边界条件填充（pad）数据，但由于数据不像图像那样高维、不需要进行压缩，没有使用池化。

有限体积方法（来自Qwen3-Max）：

有限体积方法（Finite Volume Method, FVM）是一种用于求解偏微分方程（特别是守恒律形式的方程，如流体力学中的Navier-Stokes方程、对流-扩散方程等）的数值方法。它在计算流体力学（CFD）、热传导、电磁场模拟等领域广泛应用。

核心思想：

有限体积法的核心思想是**将计算区域划分为若干个控制体积（或称控制单元）**，然后对每个控制体积应用**物理守恒定律**（如质量、动量、能量守恒），从而将偏微分方程转化为一组代数方程。

这种方法天然保证了**局部和全局的守恒性**，这是其相对于有限差分法（FDM）和有限元法（FEM）的一个显著优势。

基本步骤：区域离散化（对区域做划分）、写出积分形式的控制方程、通量近似、时间离散，求解代数系统

二、卷积编码-解码器网络

自编码器常用于非线性地降维，由两个NN构成，一个将数据从输入层转化为有限个隐藏单元（编码器），一个输出具有与输入层相同单元数的节点（解码器）。

在对随机流体的流动建模时，Zhu等人开发了一种具有物理约束的卷积编码-解码器网络和生成式模型，利用基于CNN的技术解一类高维空间变系数随机偏微分方程，效率比FC-NN高。Wang等则提出了Theory-guided Auto - Encoder (TgAE)，能够将**物理约束整合于CNN中**。Geneva和Zabaras提出了一种深度自回归密集编码器和物理约束训练算法，用于**预测瞬态偏微分方程**（含时间变量的非稳态PDE），他们将该模型扩展到贝叶斯框架，以量化认知不确定性和偶然不确定性。

2.1.3 循环神经网络

循环神经网络（RNN）是一种更先进的神经网络，与前馈神经网络的区别在于在同一个隐藏层的神经元互相连接形成一个有向的环，可以接受序列数据，适合用来解决与时间有关的任务。RNN一次处理一个输入，隐藏单元内可以维护一个状态向量用于记忆先前的输入，并输出作为下一个元素的额外输入。

可以从两方面理解RNN：首先，作为一个状态系统，其属性是除了第一种状态外，每个状态都会产生一个结果；其次，作为一个香草前馈神经网络序列（vanilla还有原始意思，vanilla FNN是一种只有单一输入输出的原始网络），每个神经网络将信息从一个隐藏层传递到下一个隐藏层。

逐层公式可以写为：

$$f_i(h_{i-1}) = \alpha(W \cdot h_{i-1} + U \cdot x_i + b)$$

其中： α 是逐元素的非线性激活函数（在RNN中常选择双曲正切函数），隐藏的状态向量 h 随隐藏层间的权重矩阵 W 演化， U 为输入层到隐藏层间的权重矩阵， b 为偏执向量。

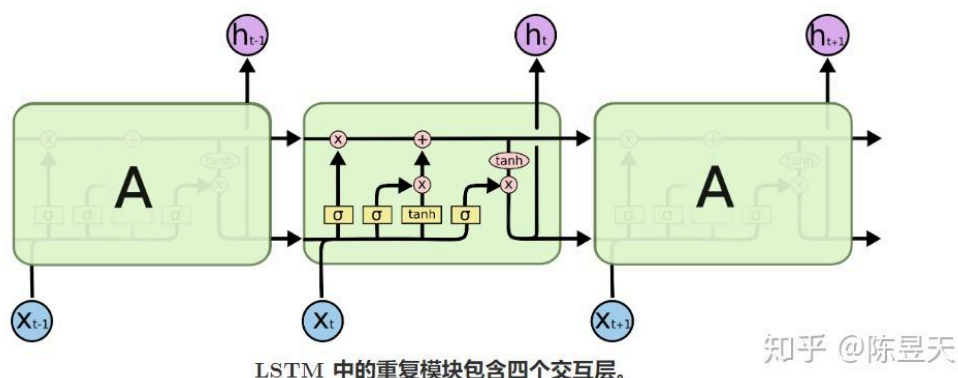
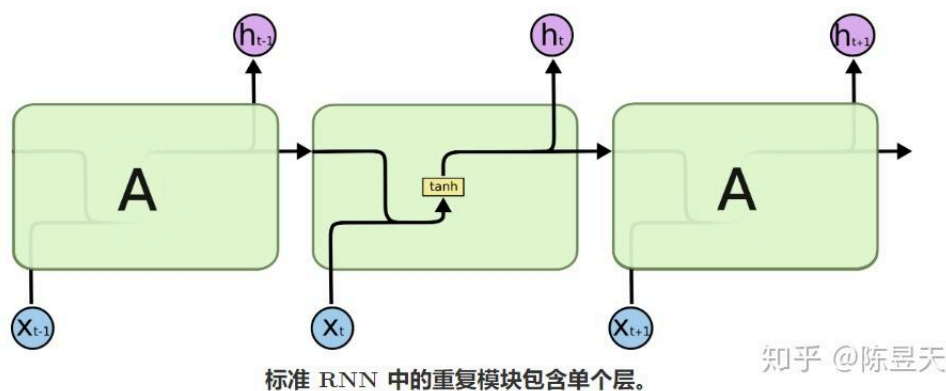
可以通过几种类型的记忆单元来增强RNN的性能，例如LSTM和GRU。

Long short-term memory(LSTM)单元内置了储存长时间信息的记忆单元（Memory Cell），可以使RNN具有长时间记忆。标准的LSTM由四个相互连接的单元：输入门、记忆门

(内部单元)、遗忘门和输出门，这些门控制了单元的状态，这个系统可以选择性地在时间序列上传播**相关的信息**以刻画动态系统中的长期依赖和短期模式。

门控循环单元 (GRU) 是为了扩展记忆而开发的另一种RNN单元；GRU与LSTM相当，但是它们包含更少的参数，更易训练。

RNN模型图解(来自知乎[LSTM - 长短期记忆递归神经网络](#))：



一、RNN架构

Viana ([RNN小节都提到他，是大牛](#)) 等人提出，在神经网络的形式下，模型的变差变为一个给定的ODE。RNN将依赖时间的变化引入了传统的前馈神经网络，使其成为对动态系统建模的理想选择。循环神经网络会周期性地转换序列中的给定状态，可以利用这一性质设计RNN单元来进行欧拉法求积分，即物理信息循环神经网络可以用于执行数值积分。

Viana等人构建了应用特定数值积分方法（欧拉，黎曼，龙格-库塔方法等）的RNN单元。RNN进而被表示为一个有向图，其中每个节点对应一个物理计算核（物理模型中的基本计算单元或算子）。为物理信息模型构建的计算图可以用来引入数据驱动节点（例如多层感知机，MLP），从而调整图中某些节点的输出，以最小化模型与真实物理系统之间的偏差

二、LSTM架构

物理学家通常采用不同的LSTM网络来描述序列到序列的输入-输出关系；但是这些网络不是同构的，不能直接相连接。Zhang等人的研究中，输入-输出关系通过一个单一的神经网络和一个基于中心有限差分滤波器的数值微分器来表示。他们展示了两种用于序列到序列的表示学习架构，从通过物理模型增强的有限数据学习特征。他们提出的网络由两到三个描述状态空间变量、非线性恢复力和滞后参数的深度LSTM网络，网络之间通过一个对状态变量求导的张量微分器（算子）连接。

另一个方法是Yucesan和Viana通过已有的欧拉法积分单元进行时间积分的架构。

2.1.4 其他PINN架构

介绍BNNs, GANs, 还有将多种PINNs结合起来的网络。

一、贝叶斯神经网络

贝叶斯框架下，Yang等人提出使用贝叶斯神经网络（BNNs）构建PINN（即B-PINNs），是一个将PDE约束作为先验知识的BNN。BNN中的权重不是一个确定的值，而是一个通过贝叶斯推断学习而来的分布。作者们使用了哈密顿蒙特卡洛方法（HMC）和变分推断来估计后验分布。Yang等人发现B-PINN中使用HMC比使用平均场高斯近似的变分推断更优。

他们也探索了使用Karhunen - Loève展开（KL）表示随机过程代替BNN。虽然KL和和BNN一样准确，而且计算更快，但是不能简单地扩展到高维情况。为了计算贝叶斯框架的后验分布，他们发现KL-HMC和深度归一化流（deep normalizing flow, DNF）都是可行的：DNF的计算开销比HMC大，但是可以在训练和更好地从目标分布中提取独立样本（快速生成独立样本）。这些特性使BNN可以用于求数据驱动的PDE解，但是只在低维情况下可用。

二、生成对抗网络

在对抗生成网络（generative adversarial networks, GANs）中，两个NN在零和博弈中与另一方竞争，一个网络生成，一个网络进行判别。生成网络接受数据并生成有现实特征的输出，判别网络将真实值和生成网络的输出进行比较，在训练之后，生成网络可以生成和真实值相差很小的新数据。

Yang等人提出一类新的GAN(PI - GANs)用于统一地解决前向、倒向和混合随机问题。经典的GANs单纯使用数据进行训练，PI - GANs则使用自动微分来将支配性物理规律以随机微分方程的形式（SDEs）嵌入到PINN架构。其中，判别网络通过一个基础的FFNN表示，生成网络则是通过SDE诱导的FFNNs和NN的组合。

三、多重PINNs (Multiple PINNs)

将多个不同的PINN结合起来的方法。

Jagtap等人在离散域上提出了一种保守的PINN (cPINN)。在这个架构中最终的结果由适当的边界条件 (interface conditions) 将每个子域中的解拼接在一起。这种分割使得网络可并行计算，这对计算效率非常重要。这种方法可以被扩展到更一般的条件，被作者称为Mortar PINN。在每个子域中可以使用不同类型的NN，用于解同一个PDE。

Stiller等人提出GatedPINN架构，将条件计算 (conditional computation) 纳入PINN。这种架构由一个gating network和一组PINNs (称为专家)，每个专家从不同的时空点求解PDE，最后在gating network整合为最终解，由gating network决定是否选用专家的解以及选好解之后的整合方式。

2.2 物理规则的注入

为了训练PDE的PINN，需要神经网络的倒数，由于选取的激活函数是光滑的，所以整个网络是可微的，那么求导就有四种方法：手动计算，符号计算（计算机求出导数表达式）、数值求导和自动微分。

手动计算显然不可取，而符号计算和数值方法（例如有限微分）在复杂函数上表现很差；自动微分 (AD) 相较数值方法可以克服了浮点数精度误差的限制，并且得到了最后导数的表达式，相较符号计算使用浮点数而不是String类型的变量，不会有估计误差，于是AD成为PINN计算导数的首选。

自动微分得到计算每个组分的计算图，通过前向和后向两个方向得到导数的数值。PINN领域大多数论文都使用AD求导，除了Fang使用局部拟合方法逼近求导算子来解PDE，同时他也证明他的PINN有收敛性。

自动微分将PDE纳入到损失函数表达式中，来自PDE的残差定义为：

$$r_{\mathcal{F}}[\hat{u}_{\theta}](z) = r_{\theta}(z) := \mathcal{F}(\hat{u}_{\theta}(z); \gamma) - f,$$

类似地，与边界条件 (BC) 或初始条件 (IC) 对应的残差神经网络通过将 $\hat{u}_{\theta}$ 代入公式 (1) 的第二个方程得到，即：

$$r_{\mathcal{B}}[\hat{u}_{\theta}](z) := \mathcal{B}(\hat{u}_{\theta}(z)) - g(z).$$

利用这些残差，可以评估近似解 u_{θ} 满足方程 (1) 的程度。对于精确解 u ，其残差满足 $r_{\mathcal{F}}[u] = r_{\mathcal{B}}[u] = 0$ 。

在 Raissi 和 Karniadakis [140]、Raissi 等人 [146] 的工作中，上述微分方程残差的原始形式被推导为：

$$r_{\mathcal{F}}[\hat{u}_{\theta}](z) = r_{\theta}(x, t) = \frac{\partial}{\partial t} \hat{u}_{\theta}(x, t) + \mathcal{F}_x \hat{u}_{\theta}(x, t).$$

在这个深度神经网络中，强物理约束被神经网络的自动微分表示，其与输入的时空坐标有关。Mathews等人将模型损失函数嵌入神经网络，然后进一步归一化为无量纲形式。使用AD的重复微分，以及用于创建偏微分方程中每个单独项的网络组合，会产生一个更大的计算图，这个计算图实际上就是PDE的近似。（这里很神奇，用来训练神经网络的结构本身就是目标的一种近似）

链式法则被用来分层地计算导数，但是有些情况下基本的链式法则不再适用，Pang等人就将其替代为分数阶导数算子的离散形式，再将其整合进入损失函数。

2.3 通过学习拟合模型

PINN和NN一样通过求使损失函数最小的参数 θ 来学习，损失函数一般由微分方程带来的损失，边界损失（含有初始条件）和数据损失项的线性组合构成：

$$\mathcal{L}(\theta) = \omega_{\mathcal{F}} \mathcal{L}_{\mathcal{F}}(\theta) + \omega_{\mathcal{B}} \mathcal{L}_{\mathcal{B}}(\theta) + \omega_d \mathcal{L}_{data}(\theta) \quad (6)$$

微分方程带来的损失常由和定义，也可以由积分定义：

$$\mathcal{L}_{\mathcal{F}}(\theta) = \int_{\Omega} (\mathcal{F}(\hat{u}_{\theta}(z)) - f(z_i))^2 dz$$

这中计算已经在PINN包（NVIDIA Modulus）中实现了，实现了更高效的积分方法。损失函数只有在有监督学习时损失函数会有预测值与实际值的偏差项。

由PDE带来的残差项 $\mathcal{L}_{\mathcal{F}}(\theta)$ ，在定义域上均匀或不均匀选取的collaboration points会加强模型和微分方程 $\mathcal{F}$ 的联系。

总的来说，损失函数来自：

- 已知的零星数据观测点 $u(z)$ （包含初始和边界条件） $\{z_i, u_i^*\}$ ，其中 $i = 1, \dots, N_d$
- 均匀或非均匀选取的collaboration points $\{z_i, r_i = 0\}$ ，其中 $i = 1, \dots, N_r$ 。

而在这个框架下优化损失函数只需要用到一般的随机梯度下降法而不需要使用约束优化法，一个经典的实现是使用均方误差计算：

$$\mathcal{L}_{\mathcal{F}}(\theta) = \text{MSE}_{\mathcal{F}} = \frac{1}{N_c} \sum_{i=1}^{N_c} \|\mathcal{F}(\hat{u}_{\theta}(z_i)) - f(z_i)\|^2 = \frac{1}{N_c} \sum_{i=1}^{N_c} \|r_{\theta}(u_i) - r_i\|^2$$

这项会对偏离PDE的点进行惩罚，有些方法中也使用积分定义的损失函数。

初始和边界条件的损失项：

$$\mathcal{L}_B(\theta) = \text{MSE}_B = \frac{1}{N_B} \sum_{i=1}^{N_B} \|B(\hat{u}_{\theta}(z_i)) - g(z_i)\|^2$$

数据点的损失项：

$$\mathcal{L}_{data}(\theta) = \text{MSE}_{data} = \frac{1}{N_d} \sum_{i=1}^{N_d} \|\hat{u}_{\theta}(z_i) - u_i^*\|^2$$

在前向问题中（因为前向问题一般是无监督学习？），该项也可能用来表示初始和边界条件；在倒向问题中该项则表示定义域中的多个解。

Raissi、Karniadakis等人最早使用以下总损失函数：

$$\mathcal{L}(\theta) = \frac{1}{N_c} \sum_{i=1}^{N_c} \left\| \frac{\partial}{\partial t} \hat{u}_{\theta}(x, t) + \mathcal{F}_x \hat{u}_{\theta}(x, t) - r_i \right\|^2 + \frac{1}{N_d} \sum_{i=1}^{N_d} \|\hat{u}_{\theta}(x_i, t_i) - u_i^*\|^2$$

$\mathcal{F}$ 中的梯度使用自动微分进行计算。模型的预测于是通过PDE约束反映了真实的物理情景，即使真实的数据点较为稀疏。

一、损失函数的性质

在诸如TensorFlow和Pytorch的机器学习包中已经实现了自动微分，可以用于计算损失函数。自动微分使得PINN可以在不需要数值离散化的（这应该就是mesh-free，无网格的）的情况下计算任意PDE和其边界条件。

同时可以控制 $\mathcal{L}_{\mathcal{F}}(\theta)$ 的权重系数 $\omega_{\mathcal{F}}$ 来保证模型的保真性（fidelity）。权重越低，保真性越低。通常未知参数的数量总是远大于测量的精确值的数目，因此DNN的训练需要正则化。

如果去掉 $\mathcal{L}_{\mathcal{F}}(\theta)$ ，就是在不知道底层控制的情况下训练网络，只满足初始和边界条件，其实就是一个NN而不是PINN。

在整个定义域上进行训练会使得优化过程的计算开销很大，因其与残差点数量有关，同时残差点位置（分布）的选取也是一个很重要的设计因素。

提升网络的表达能力（**深度和神经元个数**），但是可能会过拟合。通过调整超参数（学习率、迭代次数等）解决该问题。

可以在激活函数里增加额外的参数调整激活函数的斜率从而提高训练速度，同时在优化过程中这些激活斜率也可以加入损失函数中。

二、软和硬约束

约束（BC）可以作为惩罚项（软约束）或者被编码进神经网络的结构中（硬约束）。常用软约束，有两个缺点：

1. 软约束不一定会被满足
2. BC惩罚项前的系数会影响学习的效率，但是没有现存理论告诉我怎么取定这个权重

Zhu等人将迪利克雷条件作为硬约束，通过NN的特定结构满足了约束。

损失函数可以使用collaboration points、galerkin方法导出的加权残差或**基于能量**来构建。Li等人比较了纯数据驱动、基于PDE的残差和基于能量的损失函数，基于PDE的残差和基于能量的损失函数各有优缺点，前者需要更多超参数，后者对大小和分辨率（精度）更敏感，但计算效率更高。

三、优化方法

大多使用minibatch sampling using Adam and the limited-memory Broyden – Fletcher – Goldfarb – Shanno (L-BFGS) algorithm（一种半牛顿优化算法）。在有噪声数据时，增加样本量和仅使用L-BFGS进行训练就可以有最优学习效果（Mathews等）。

对于一个损失函数可能有多个局部最小值，优化过程至少能获得其中一个，寻找全局最优是一个NP-hard问题。

Zhu等人采用了结合自适应学习率和动量方法的Adam方法来提高收敛速度，因为随机梯度下降（SGD）很难管理随机collocation points，特别是在3D情景中。

Yang等人使用带有梯度惩罚的Wasserstein GANs (WGAN-GP)，并证明它们比普通GANs更稳定，特别是在拟合具有确定性边界条件的随机过程时。

cPINN[71]基于对每个子域解规则性的直观认识，**灵活地**选择网络超参数，如优化方式、激活函数、网络的深度和宽度。例如，对于光滑区域，可以使用浅神经网络，而在假设具有复杂性质的区域可以使用深度神经网络。

He等人提出了两步训练法：先用预定义停止条件的Adam算法最小化损失函数，再用L-BFGS-B优化。在训练数据、残差点较少的时候，L-BFGS-B有较快的收敛速度和较小的计算开销。

两个实例：在DeepONet训练中使用无量纲和归一化数据来提高稳定性。此外，DeepM&Mnets尝试解决无量纲形式的控制方程，包括斯托克斯方程，电势和离子输运方程。

在训练过程初始化（参数的最初选取）方面，初始化做得越好，收敛速度越快。但Pang等提出了一种选择最合适初始化的技术。通过使用有限次数的迭代，可以首先解决逆问题。由于离散化、采样和优化误差，该初步解的精度较低。从低保真度问题中得到的优化参数可以作为合适的初始参数。

2.4 PINN的学习理论

在用数值方法求PDE时我们关注解的稳定性、一致性和收敛性。数值离散方法的误差可以被一致性和稳定性约束。The Lax - Richtmyer Equivalence Theorem表明当有一致性和稳定性时，大致保证有收敛性。

学界试图用同样的范式评价PINN。PINN的收敛性和稳定性与神经网络从物理定律和数据中学习的程度有关。

2.4.1 PINN的一致性

主要是对NN估计出来的函数与真实函数之间的差异进行分析。

考虑一个神经网络配置，其系数在向量 θ 中复合，基数等于神经网络的系数数量数 $\#\theta$ 。在这种情况下，我们可以考虑假设类 $\mathcal{H}_n = \{u_\theta : \#\theta = n\}$ ，NN的学习能力和参数的数量 n 有关。

在这样的配置下，问题在于估计而来的函数和物理世界中实际的函数是否是一致的，即 n 趋向于无穷时估计函数 $\hat{u}_\theta$ 是否会趋向 u ？

在使用双曲正切作为激活函数时，De Ryck等人证明了两者之间的差异随着NN宽度的增加而减小，宽度增加到正无穷时差异趋近于0。

即使假设类 $\mathcal{H}_n = \{u_\theta : \#\theta = n\}$ 含有PDE的精确解，我们也不能确保优化过程会使得最后的结果一致。有作者证明了函数列随 n 强收敛于线性二阶椭圆型抛物型PDE的解。

2.4.2 统计学习误差分析

考虑PINN作为统计学习，可以使用统计学习的误差分析的方法，有三个要素：优化、泛化

误差、估计误差 (approximation error, 和整体架构有关)。

设 N 个配置点位于 $\Omega = \Omega \cup \partial\Omega$ 上, 一个由参数 θ 实现的神经网络近似值记为 $\hat{u}_\theta$, 在点 z_i 处评估, 其精确值为 h_i 。根据 Kutyniok 的符号, 经验风险定义为:

$$\widehat{\mathcal{R}}[u_\theta] := \frac{1}{N} \sum_{i=1}^N \|\hat{u}_\theta(z_i) - h_i\|^2 \quad (7)$$

和2.3中定义的误差有关。

也可以取连续形式的 $\hat{u}_\theta$ 的风险, 计算公式如下:

$$\mathcal{R}[\hat{u}_\theta] := \int_{\hat{\Omega}} (\hat{u}_\theta(z) - u(z))^2 dz \quad (8)$$

其中, 近似器 $\hat{u}_\theta$ 与解 u 之间的距离是通过 L^2 -范数获得的。经过 DNN 训练过程后, 由 PINN 计算得到的最终近似值为 $\hat{u}_\theta^*$ 。误差分析的主要目标是找到预测 u 的风险的合适估计值 $\mathcal{R}[\hat{u}_\theta^*]$ 。

优化过程则是通过梯度优化方法最小化一个一般非凸的函数, 结果实际上并不总是全局最优, 所以这里也会产生一个优化误差 optimization error:

$$\varepsilon_O := \widehat{\mathcal{R}}[\hat{u}_\theta^*] - \inf_{\theta \in \Theta} \widehat{\mathcal{R}}[u_\theta]$$

由于第二项是未知的, 优化误差也是未知的。使用基于梯度的优化方法通常涉及各种工程方法和耗时的微调。也常用随机梯度下降方法, PINN常用Adam和L-BFGS。PINN适不适合梯度相关的优化方法仍待解决。

在未见数据的准确性称为泛化误差:

$$\mathcal{E}_A := \inf_{\theta \in \Theta} \mathcal{R}[u_\theta]$$

Shin等人最早研究泛化误差的一致性。

Kutyniok发现, 估计函数和精确解之间总误差由上述三种误差控制:

$$\mathcal{R}[\hat{u}_\theta^*] \leq \varepsilon_O + 2\varepsilon_G + \varepsilon_A \quad (9)$$

2.4.3 对PINN误差分析的结果

两层隐藏层、双曲正切为激活函数的NN，用 Sobolev 空间中的一个界来逼近函数 u ：

$$\|\hat{u}_{\theta_N} - u\|_{W^{k,\infty}} \leq C \frac{\ln(cN)^k}{N^{s-k}}$$

其中 N 是训练点的数量， $c, C > 0$ 是与 N 无关且明确已知的常数， $u \in W^{s,\infty}([0, 1]^d)$ 。我们注意到，该神经网络 (NN) 的宽度为 N^d ，且参数 θ 同时依赖于训练点数量 N 和问题的维度 d 。

对泛化误差的分析常针对特定PDE：

作者	PDE 类型	工作内容
Shin 等	线性二阶椭圆型和抛物型 PDE	提供了 PINN 的收敛性估计
Shin 等	所有线性 PDE（包括双曲型方程）	将收敛性结果推广至更一般的线性问题
Mishra 和 Molinaro	一般前向 PDE 问题	建立了 PINN 的抽象框架，通过训练误差和训练点数量估计泛化误差
Mishra 和 Molinaro	逆问题	将上述抽象框架扩展应用于 PDE 逆问题

表格来自Qwen-max

De Ryck et al针对纳维斯托克斯方程提出估计误差小则泛化误差小：

$$\mathcal{R}[\hat{u}_\theta] = \|u - \hat{u}_\theta\|_{L^2} \leq \left(C\widehat{\mathcal{R}}[u_\theta] + O\left(N^{-\frac{1}{d}}\right) \right)^{\frac{1}{2}}.$$

需要通过修改某项来获得更好的估计，需要更多的训练点和更大的模型，规模呈指数增加 (curse of dimensionality , CoD)

De Ryck和Mishra证明：对于Kolmogorov型PDE（即热方程或black - scholes方程），以下不等式几乎总是成立：

$$\mathcal{R}[\hat{u}_\theta] \leq \left(C\widehat{\mathcal{R}}[u_\theta] + O\left(N^{-\frac{1}{2}}\right) \right)^{\frac{1}{2}},$$

且与维数d无关。

Mishra和Molinaro研究了高维的辐射传递方程，证明了泛化误差受训练误差和训练点数量的限制，并且维度依赖于一个对数因子：

$$\mathcal{R}[\hat{u}_\theta] \leq \left(C\widehat{\mathcal{R}}[u_\theta]^2 + c\left(\frac{(\ln N)^{2d}}{N}\right) \right)^{\frac{1}{2}},$$

在这个问题中PINN没有维数诅咒，不依赖于维数而只依赖于训练点数量

三、用PINN解偏微分方程

前面这一段和introduction内容差不多，PINN可以用来解常微分方程、偏微分方程、积分-微分方程、随机微分方程，值得学习的是如何写综述：

本章将从ODE开始，接着讲稳态偏微分方程，再是纳维斯托克斯方程，紧接着是一些动力系统问题，最后讨论所有这些PDEs都可在相应的贝叶斯问题中被解决，最后这小节提供了克服随机微分方程中不确定性的方法论

这是一种由简单到繁复知识的综述方法。

在这章笔记将不会特别记录用PINN解了什么方程，而是关心谁针对方程的什么特性对PINN进行了什么样的调整从而解决了什么问题。

3.1 常微分方程

PINN被用于ODE的structural identification（模型参数的确定等），使用Neural ODEs，其可以被视为残差网络的连续表示， f 表示被 θ 参数化的NN，则动态系统为：

$$\frac{du(t)}{dt} = f_0(u(t), t); u(t_0) = u_0$$

Neural ODEs在学习参数 θ 时分为物理信息项和未知差异项两部分。

Zhang等人使用**LSTM网络**求解在地震作用下表现出强非线性行为的结构系统：

$$\ddot{u} + g = -\Gamma a_g$$

有向图模型可以直接将ODE转化为深度神经网络，同时使用欧拉RNN进行数值积分。

Tong等人利用ODE网络的思想创建了Symplectic Taylor neural networks，由两个子网构成，使用它们使用基于用欧拉方法计算的残差块**辛积分器**（symplectic integrators），而非最初在ODE-net中使用的RungeKutta。

3.2 稳态偏微分方程

稳态偏微分方程是一类与时间无关的偏微分方程，即不会发生演化，数学表达为：

$$\begin{aligned}\mathcal{F}_s(u(x); q) &= f(x), \quad x \in \Omega, \\ B(u(x)) &= 0, \quad x \in \partial\Omega\end{aligned}$$

特别地，椭圆型方程可以写为：

$$\mathcal{F}_s(u(x); \sigma, \mu) = -\operatorname{div}(\mu \nabla u) + \sigma u$$

Tartakovsky等人描述扩散系数K未知的线性/非线性扩散方程：

$$\begin{aligned}\mathcal{F}_s(u(x); \sigma) &= \nabla \cdot (K(x) \nabla u(x)) = 0 \\ \mathcal{F}_s(u(x); \sigma) &= \nabla \cdot [K(u) \nabla u(x)] = 0\end{aligned}$$

用于表示均匀多孔介质中的非饱和流动，其中u为水压力，K(u)为多孔介质的电导率。直接测量K(u)是困难的，因此Tartakovskiy等假设u的备选估计只有有限个。他们证明了PINN方法优于最先进的后验概率方法，并且**仅利用capillary pressure data**，PINNs就可以估计非饱和流动的压力传导性。

Kharazmi等人提出了变分PINN (VPINN)，优点是**通过分部积分法降低微分算子的阶数**。

Haghighat等采用**非局部**方法和PINN框架来求解线弹性和弹塑性变形的二维准静态力学问题。他们定义了**弹塑性的损失函数**，前馈神经网络接受位移输入，输出应变张量和应力张量的分量。解中的**局部变形和强梯度**会使边值问题难以求解。Haghighat等人将动态微分算子(PDDO)纳入非局部方法和PINN范式，并证明PDDO框架可以使用全局函数捕获应力和应变集中点。

Dwivedi和Srinivasan结合PINN和ELM(Extreme Learning Machine)构建了PIELM，只考虑了线性微分算子。

Ramabathiran和Ramachandran针对线性椭圆方程，想要解决**解中的非光滑性**，重新解释PDE解的无网格表示提出了一类**部分可解释的**稀疏神经网络架构 (SPINN)。

EikoNet被用于解决三维的**Eikonal equations**：

$$\|\nabla u(x)\|^2 = \frac{1}{v^2(x)}, \quad \forall x \in \Omega,$$

但是EikoNet仅对单个给定速度模型有效，因此即使稍微改变速度，也需要**重新训练**神经网络，EikoNet本质上预测了从震源位置到接收位置所需的时间。

PINN在测试精度上优于the first-order fast sweeping solution，特别是在**各向异性**模型中。

在另一篇文章中，模型需从电解剖图中学习心脏组织纤维方向的合成和患者数据，用文章用各向异性Eikonal方程建模，损失函数中加入了**电导率矢量的总变差正则化项**。

忽略各向异性，Sahli Costabalet等解决了心脏激活映射问题（cardiac activation mapping），其中PINN与**随机先验函数**一起用于量化数据不确定性，并创建自适应采样策略以获取和创建激活图。

3.2.2 非定常偏微分方程（Unsteady PDEs）

和稳态偏微分方程相对，考虑了时间演化的偏微分方程。PINN在解这类方程时具有**可靠性**和**灵活的方法**。

3.2.2.1 对流-扩散-反应问题

这类问题的数学表示：

$$\begin{aligned}u_t &= \mathcal{F}_x(u(x)), \quad x \in \Omega, \\B(u(x)) &= 0, \quad x \in \partial\Omega\end{aligned}$$

扩散问题：

对复合材料的热转移过程建模：

$$\frac{\partial T}{\partial t} = a \frac{\partial^2 T}{\partial x^2} + b \frac{d\alpha}{dt}$$

a,b是参数， α 表示固化程度取值在0到1之间，内部热源项正比于 $d\alpha/dt$ 。

Amini Niaki提出了一种由两组不相连的子网构成的PINN，通过一种**顺序训练算法**自适应**权重和损失**，提高模型的预测准确率。基于物理观察，在描述温度的网络最后一层选用有**正输出且可导、导数非零的激活函数**——**Softplus函数**（ $\text{Softplus}(x)=\log(1+e^x)$ ，是ReLU函数的连续近似）；在描述固化程度（cure）的网络最后一层使用Sigmoid函数作为激活函数；在隐藏层使用双曲正切函数。该网络正确预测了目标。

Stefan问题由两部分组成，一是传统形式——确定相变期内一个域内的温度分布，二是反问题——由一组自由边界条件（即Stefan条件）确定。Cai等人构建了一个PINN：用一个DNN表

示未知边界，一个FCNN输出两个相的表示；用此PINN将温度分布参数化。这导致了来自三个神经网络的共三部分的残差。两个网络的参数 θ 和 β 由均方误差函数优化：

$$\mathcal{L}_{\mathcal{F}}(\theta) = \mathcal{L}_r^{(1)}(\theta) + \mathcal{L}_r^{(2)}(\theta)$$

带入两个相的热方程：

$$\mathcal{L}_r^{(k)}(\theta) = \frac{1}{N_c} \sum_{i=1}^{N_c} \left\| \frac{\partial u_{\theta}^{(k)}}{\partial t}(x^i, t^i) - \omega_k \frac{\partial^2 u_{\theta}^{(k)}}{\partial x^2}(x^i, t^i) \right\|^2, \quad k = 1, 2.$$

这个损失函数是在被选中的collocation locations上训练的， ω_1, ω_2 是训练权重参数。初始和边界条件带来的残差：

$$\mathcal{L}_B(\theta) = \mathcal{L}_{sbc}^{(1)}(\theta, \beta) + \mathcal{L}_{sbc}^{(2)}(\theta, \beta) + \mathcal{L}_{snc}(\theta, \beta) + \mathcal{L}_{s0}(\beta)$$

对于具体数据点，残差具体为：

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} \|u_{\theta}(x_{\text{data}}^i, t_{\text{data}}^i) - u_i^*\|^2$$

Cai等人利用这个框架找到了准确解，而一般的PINN因为局部最小缺陷不能在反问题中确定未知的热扩散值。对此，可以采用了**动态权重技术**，减轻了在梯度流动动力学中由于**刚度**而引起的问题。

平流问题

对流-弥散方程：

$$u_t + \nabla \cdot (-\kappa \nabla u + \nu u) = s$$

二维对流-弥散方程：

$$u_t + \nabla \cdot (-\kappa \nabla u + au) = 0$$

对此问题，PINN对边界误差比ELM方法更好，可能因为He and Tartakovsky给初始和边界条件的权重较大。

对一个有**极大梯度**的PDE和其线性平流方程，PINN和PIELM可能失效。

Schiassi et al比较了Deep-TFC和X-TFC，前者的准确性在问题较复杂时预测更准确。

在二维Burgers方程中，Jagtap说明通过**获得激波shock位置**的先验信息，cPINN因其灵活的域划分方式可以在划分域中**捕捉到迅速下降**的特征。

而Arnold和King 则在**模型预测控制**的背景下解决了Burgers方程。

对含长时间积分二维扩散反应方程，Meng等人使用PPINN将时间域等分，其中PPINN由一个**粗粒度（CG）求解器**和一个高精度的PINN**精细求解器**构成。

3.2.2.2 流问题

非定常偏微分方程的特例，与流体的运动有关。其中很有名的是纳维斯托克斯方程（**N-S方程，世纪难题之一**）。传统方法解起来各有问题，PINN因其**无网格**的解法有可能克服这些问题。

N-S方程

$$\mathcal{F}_x(u(x); v, p) = -\text{div}[v(\nabla u + \nabla u^T)] + (u + \nabla)u + \nabla p - f,$$

u是流速，p是压力，v是粘度

$$\text{div}(u) = 0$$

Burgers方程是N-S方程的特例。

Arthurs and King的PINN使用**有限元数据**进行训练，估计整个参数域的Navier-Stokes压力和速度场。他们提出了一种**主动学习算法（ALA）**，同时结合了结合域和网格生成器以及传统的带PINN的PDE求解器，用于训练预测大面积参数空间上PDE解的PINN。

接下来作者罗列很多形式的方程和学者们对相应PINN的改进，一般是用到之前不同的网络类型、不同的数据集、不同的优化方法、不同的边界约束方法来提高稳定性和准确性，流体方面我是在知之甚少，就不一一罗列了。

在无数据的学习过程中，边界的硬约束可能比软约束更好。

双曲方程

双曲守恒率在血液动力学常用于简化NS方程。

Abreu and Florindo研究了无粘非线性Burgers方程和一维buckley - leverett两相问题，具有以下形式：

$$\frac{\partial u}{\partial t} + \frac{\partial H(u)}{\partial x} = 0, \quad x \in \mathbb{R}, \quad t > 0, \quad u(x, 0) = u_0(x),$$

欧拉方程符合双曲守恒律，允许不连续解的存在。用数值方法捕捉不连续流的特征是很有挑战性的，Mao等人探索了一维欧拉方程，有一个比较精确的结果‘并发现：在高梯度区域周围适当地聚类训练数据点可以提高该区域的解精度，并减少整个区域的误差。表明在具有高梯度解的区域中可以使用单个局部网络，从而开发出具有不同特征的局部PINN组合，这些特征符合每个子域中已知解的先验知识，cPINN就可以实现这一点。

Mao等人在不连续点周围仅仅取了零星几个点就得到了解（这个方法很强大）

3.2.2.3 量子问题

一维非线性Schrödinger方程：

$$i \frac{\partial \psi}{\partial t} + \frac{1}{2} \frac{\partial^2 \psi}{\partial x^2} + |\psi|^2 \psi = 0 \quad (10)$$

是复数解

量子谐振子：

$$i \frac{\partial \psi(x, t)}{\partial t} + \frac{1}{2} \Delta \psi(x, t) - V(x, t) = 0$$

对这个问题，Stiller等人提出用一个门控网络来决定使用哪个MLP（tanh为激活函数），解是MLP结果的加权和，结果与最先进的求解器相当。

3.3 其他应用

3.3.1 分数阶微分方程

分数阶方程用于模拟现象学性质，其中参数需要从实验数据中获得，但是在时空域中，实际测量通常稀缺，并可能受到噪声影响。

自动微分不能用于分数阶算子，所以PINN的架构在这类问题里要更复杂，可以使用L1 scheme。

Pang等人通过生成fpinn来识别总体形式已知但系数和算子未知的分数阶偏微分方程的参数，用混合技术构建损失函数，对**整数阶算子自动微分**，对**分数阶算子用数值离散**。

Kharazmi扩展了Pang的方法，使用离散NN表示分数阶算子，用一个较大的NN表示状态。

3.3.2 不确定性估计

PINN不确定性的来源很多，训练数据很影响准确性。

[B-PINN](#)已经在上文介绍过了，与PINN不同的是可以引入噪声数据并**量化噪声**。

Yang et al考虑与时间独立的椭圆型随机微分方程，需要估计三个随机过程：解 $u(x; y)$ ，强迫项 $f(x; y)$ ，严格大于0的扩散系数 $k(x; y)$ ，i.e.

$$\begin{aligned}\mathcal{F}_x[u(x; y); k(x; y)] &= f(x; y), \\ \mathcal{B}_x[u(x; y)] &= b(x; y),\end{aligned}$$

他们进一步考虑了有限数量sensor测量的结果。当对 f 的sensor足够时，随着对 k 的sensor减少，对 u 的sensor增加，问题在这个过程中逐渐从一个正向问题变为混合问题最后变为逆问题。

Cavanagh et al使用**核密度估计 (KDE)**将形态空间嵌入概率密度函数，在训练PINN时使用贝叶斯计算和序列蒙特卡洛方法训练参数。

3.4 用PINN解微分问题的实例

本节介绍一个实例：一维非线性Shrödinger问题 (NLS)，PINN可以**处理周期性边界条件和复数解**的问题。

在域 $\Omega = [-5, 5] \times [0, T]$ 内写为

$$\begin{cases} i\psi_t + 0.5\psi_{xx} + |\psi|^2\psi = 0 & (x, t) \in \Omega \\ \psi(0, x) = 2 \operatorname{sech}(x) & x \in [-5, 5] \\ \psi(t, -5) = \psi(t, 5) & t \in (0, T] \\ \psi_x(t, -5) = \psi_x(t, 5) & t \in (0, T] \end{cases}$$

Raissi等人用传统光谱模拟方法创建了一个高分辨率的数据集，基于Chebfun的开源软件用MATLAB对这个方程积分。PINN在初值数据 $\{x_0^i, \psi_0^i\}_{i=1}^{N_0}$ 、边界collocation数据 $\{t_b^i\}_{i=1}^{N_b}$ 和域内的collocation points $\{t_c^i, x_c^i\}_{i=1}^{N_c}$ 上训练，这些点从高分辨数据上随机抽取。

NN架构：输入位置 x 和时间 t ，输出解的实部和虚部。

这个问题不是逆问题，没有数据点上的损失，损失函数的两项为：

$$\mathcal{L}_B = \frac{1}{N_0} \sum_{i=1}^{N_0} |\psi(0, x_0^i) - \psi_0^i|^2$$

$$+ \frac{1}{N_b} \sum_{i=1}^{N_b} (|\psi^i(t_b^i, -5) - \psi^i(t_b^i, 5)|^2 + |\psi_x^i(t_b^i, -5) - \psi_x^i(t_b^i, 5)|^2)$$

$$\mathcal{L}_F = \frac{1}{N_c} \sum_{i=1}^{N_c} |\mathcal{F}(t_c^i, x_c^i)|^2.$$

使用**拉丁超立方抽样** (Latin Hypercube Sampling, LHS) 抽取采样点，放在在训练神经网络之前的基准数据中。

使用Adam优化方法，选取LBGFS调参的学习率。

使用Stiller的代码实现，Raissi提供的基准解，尝试深度不同、宽度不同的网络，分析其均方偏差和平均绝对误差。

拟合得最好的PINN很难预测 $(x, t) = (0, \pi/4)$ 周围的中心高度，以及 $t \in (\pi/4, \pi/2)$ 中与时间间隔 $t \in (0, \pi/4)$ 对称的映射值。

四、PINN的数据、应用和软件

这一章有些叙述和introduction差不多，以及比较专业的知识，不赘述、不班门弄斧。

4.1 数据

使用PINN需要对待解的方程和常见NN架构有充分的了解（也是这篇综述的叙述目的和思路）。

神经网络的结构显然对结果有影响，PINN比较有特色的问题在于选取训练点，可以随机选取，但是有些问题中采样分布的影响很大。

PINN的另一重要性质：具有**描述潜在非线性变量的能力**。Zhang等人的研究发现如果缺失一个变量的测量，PINN仍然能够准确估计这个变量。

4.2 应用

血液动力学

主要和NS方程有关，用于模拟血液流，还有一些工程和生物方面的磁场和电场。PINN可以快速适应病人的新状态，得到了和实际情况一致的输出。

流问题

有减少实际实验次数的潜力，可迁移到其他跨学科场景

Mao等人用PINN解决传统方法无法解决的反问题。

传统PINN在训练时间范围之外预测表现很差，Kim等人提出了一种全新的方法解决这一问题

光学与电磁应用

分子动力学和材料相关应用

地球科学与弹性静力学问题

工业应用

4.3 软件

五、PINN未来的挑战与方向

PINN面对真实的情况和不同的方程仍有问题待解决：理论问题（一致性和稳定性等问题）和实现问题（如何处理边界条件，选取怎样的NN结构，优化问题等）。有些数值方法是针对特定PDE的，它们的表现总比PINN好，PINN在这些问题中**不能学习其中复杂的物理现象**（比如表现出多尺度、混沌或湍流行为的解）。

5.1 解决PINN的理论问题

最重要的问题：估计出来的解是否和真实解一致。

研究表明初始隐藏层可能负责编码低频分量（表示低频信号需要**更少的点**），随后的隐藏层可能负责表示高频分量。这可能是**频率原理**的扩展，原理表明：DNN在训练过程中从低频拟合到高频拟合目标函数，表明DNN可能存在低频偏差，也解释了DNN在随机数据集上不能很好地泛化的原因。对于PINN，**大规模特征首先出现，而小规模特征则需要多个训练周期迭代**。

初始化损失函数，特别是在泛化误差上是一个研究重点。许多理论结果都是基于正交规则，在随机和同分布的点上进行流量估计（**不在这些条件上又会有什么结果？**）。选取配置点（collocation points）的技术、**动态损失加权法**都需要进一步研究。

由于物理信息的加入，优化方法的选取需要基于优化和数值分析、动态系统理论的支持。关键问题是理解PDE刚度与梯度下降算法对PINN的影响之间的关系。

PINN不受维度灾难的影响的原因也值得研究。根据已发表PINN论文，它们可以独立于问题规模轻松扩展。随着问题维度的增加，PINN的计算成本并不会呈指数级增长；这种特性在神经网络架构中**普遍存在**，但目前尚无对此类现象的**正式解释**。最近，Bauer和Kohler证明了基于前馈神经网络（FNN）的最小二乘估计可以在**非参数回归**中避免维度灾难。

在深度学习框架下，考虑NN可以拟合的函数空间，对应一个PDE的解空间是否和这个函数空间一样，也是一个值得考虑的问题。

由于PDE的软约束，PINN可能无法拟合一个解。

5.2 优化PINN实现

待研究的方面：

如何针对物理问题配置最优的PINN模块组合——非全链接神经网络是否适用于PINN，对PINN性能的影响如何？PINN特别适用于什么特别的网络结构（例如**傅里叶神经算子**、正弦表示网络SIRENS）？NN的网络深度和宽度是否能改善PINN的性能，如果可以，是否存在一个**最小规模**使得小于此规模的网络无法正确学习物理情景？适用PINN的激活函数？对不同微分方程采用什么微分方法训练网络（Chiu等人提出了结合数值和自动微分的方法，效率更高更准确）？

损失函数的配置——**所有损失项应具有相同数量级**，否则较小数量级的项会成为无效惩罚；目前没有客观方法确定各损失项的权重；没有机制确保损失函数最后会被优化到容差中（收敛）。

PINN的优化方法——没有一般的Adam和BFGS算法之外特化于PINN的优化方法。

学习率、梯度归一化技术（动态分配权重消除损失项的主导作用）对PINN的影响；PINN的误差分析。

5.3 SciML中的PINN

PINN取代传统数值方法的困难：在对流或粘性系数高时，PINN无法学习其中的规律。但当问题表现为Seq2Seq的学习任务而不是直接求解整个时空时，误差降低了。这些问题需要更深刻地探索科学问题和ML方法之间的关系。（也许就是AI4Science?）

PINN在一些问题中比传统数值方法（例如有限元，以线性方式近似）更好，因其以非线性方法拟合函数，缺陷在于训练NN比数值方法更耗时。

从二维扩展到三维时，NN的规模需求变大，需要更大的batch size，训练时间也会增加。在利用PINN求解数学模型时，应该要对问题归一化。（无量纲更好处理）

PINN难以预测训练时间之外的时间状态，Wang等人提出解决方案：重新设计损失函数，考虑物理因果。另一方法是使用集合一致性作为纳入新点的标准，基本思想是远离预测时间越远，发生错误的可能越大。

PINN在多尺度建模中仍缺乏应用。

5.3 AI框架中的PINN

对于更多应用场景可以考虑将PINN框架纳入深度神经网络强化学习中，使智能体通过实验更好地理解环境，获取更高维的因果关系、进行因果推理。PINN可以作为智能体嵌入强化学习框架，类似迁移学习范式。

PINN也可以视为深度学习与符号智能融合的实例。符号范式认为智能是通过操作表示和交互的抽象模型产生的。这个模型可以通过逻辑推理学习特征，但与世界数据有差异，深度学习则和世界数据拟合得很好。PINN有将两者结合潜力。

因果模型是物理模型和统计模型的中间体。一方面，微分方程模型预测了物理系统的未来行为、估计扰动的影响、预测变量间的统计依赖；另一方面，统计模型通常不涉及动态过程，而是通过一些变量预测在相同条件下的其他变量。融合ML和图因果表示论是一个新型课题，PINN可以在混合（混合前行和反向）问题时确定因果关系。（因果推断比较有前景，融合物理模型的NN表示应该很有意思）

六、 读后总结

这是我第一次如此详细认真的精读完一篇英文文献，此前参加的一些项目知识走马观花地看了些文章。文章先介绍PINN的基本概念，再按照人工智能领域“数据、模型、优化、结果”的整体思路介绍了PINN常见的网络结构、模型相关的数学问题，接着介绍有什么类型的微分方程应用过上述的PINN，最后陈列出一系列应用和PINN的未来前景。

我主要学习了常见的网络架构和在解微分方程这个特定的应用场景下学者们对各个组件的调整，尤其是损失函数的建构。阅读本文时，我甚至想到曾经读文献见过的元胞自动机——元胞

自动机可以在网格上模拟微分方程，根据给定的更新规则随离散时间演化，这正类似于非定常偏微分方程，那么是否可以用神经网络训练出一组最符合目标方程的全局演化规则/单个元胞规则？当然，类似于本文提到的评价模型需要一致性、稳定性、收敛性等数学基础，对元胞自动机自然有类似的疑问：一个微分方程能否由全局更新规则表示，误差是否会收敛，对更新规则的数量是否有要求，更新规则的数量是否会增强模型的拟合度，如果可以，是否存在一个最低数量的演化规则……在引擎检索确实能搜索到相关结果，例如Physics-Informed Graph Neural Cellular Automata，甚至在顶刊Light: Science & Applications也有一篇Deep learning with photonic neural cellular automata！在这个过程中我自然获得了极高的成就感。

除了神经网络和微分方程方面的知识，我也学习到了综述的写作方法：对于这类将某种技术应用到某类问题的综述，应该说明为什么要将这个技术应用到这类问题（与先前解决这类方法有什么优势），再介绍技术主要类型和原理，以及应对这类问题时技术需要做出怎样的调整，接着考察技术如何应用在不同类型的技术上，辅以一个实例，最后介绍实际的应用、代码实现、和未来前景。

最后，我在第一章的问题也在阅读过程中解决了：

物理信息如何注入？

答：物理信息作为软约束时加入损失函数注入到模型中，硬约束时则需要改变神经网络的架构

具体的一个PINN也可以在第三章找到。

AI使用情况

阅读文献时使用了夸克浏览器的划词翻译功能。

本读书笔记基于思源笔记，公式由tex编译，所以也使用了千问的图像识别转公式功能：

接下来我将会给你一系列图片，图片将会包含公式，请你将公式翻译成可复制到思源笔记中的tex代码，尤其注意应该用美元符号包裹公式并放在markdown代码框中，明白请回复1

Qwen3-Max 16:19:51

1

`</>` 代码 |     ...

$$\frac{du(x,t)}{dt} = f(u(x,t),t)$$

同上

Qwen3-Max

Markdown

 编辑   

```
1 $$
2 \frac{du(x,t)}{dt} = f(u(x,t),t)
3 $$
```

    ...



Finally, since the initial condition can actually be considered as a type of Dirichlet boundary condition on the spatio-temporal domain, it is possible to denote $\mathcal{B}$ as the operator indicating arbitrary initial or boundary conditions related to the problem and g the boundary function. Indeed, the boundary conditions can be Dirichlet, Neumann, Robin, or periodic boundary conditions.这段话什么意思

Qwen3-Max

这段英文的意思是：

最后，由于初始条件实际上可以被视为在时空域（spatio-temporal domain）上的一种狄利克雷边界条件（Dirichlet boundary condition），因此我们可以用符号 $\mathcal{B}$ 来表示与该问题相关的任意初始条件或边界条件的算子，而用 g 表示对应的边界函数。事实上，这些边界条件可以是狄利克雷（Dirichlet）、诺依曼（Neumann）、罗宾（Robin）或周期性边界条件（periodic boundary conditions）。



详细解释：

边界条件的prompt

Qwen3-Max 20:18:26

以下是 **Sparse Neural Networks**（稀疏神经网络）和 **Extreme Learning Machine**（极限学习机，ELM）的清晰解释，包括定义、原理、特点与对比：

1. Sparse Neural Networks（稀疏神经网络）

✅ 定义：

稀疏神经网络是指**权重矩阵或连接结构中大部分元素为零**（即“稀疏”）的神经网络。这种稀疏性可以出现在：

- **权重稀疏**：训练后剪枝，使许多权重为 0；
- **结构稀疏**：预先设计连接模式（如只连接局部区域），如卷积网络本身具有稀疏连接特性。

💡 核心思想：

- 并非所有神经元之间都需要连接；
- 通过移除冗余连接，在**不显著损失性能的前提下**，降低模型复杂度、计算量和内存占用。



稀疏网络和极限学习机的prompt

Qwen3-Max ▾



“拉普拉斯算子可以被有限体积方法离散化”，其中有限体积方法（a finite volume approach）是什么意思

Qwen3-Max

“拉普拉斯算子可以被有限体积方法离散化”中的 **有限体积方法**（Finite Volume Method, FVM）是一种广泛应用于计算流体力学（CFD）、传热、电磁场等领域的**数值离散技术**，特别适合求解**守恒型偏微分方程**（如质量、动量、能量守恒方程）。

✅ 一、有限体积方法的核心思想

将计算区域划分为若干个“控制体积”（control volumes），

然后对每个控制体积**积分守恒律**，

从而将偏微分方程转化为关于**单元平均值**的代数方程。

🌐 物理意义：



有限体积方法

Formal findings for generalization errors in PINN are provided specifically for a certain class of PDE. In Shin et al [157] they provide convergence estimate for linear second order elliptic and parabolic type PDEs, while in Shin et al [158] they extend the results to all linear problems, including hyperbolic equations. Mishra and Molinaro [113] gives an abstract framework for PINN on forward problem for PDEs, they estimate the generalization error by means of training error (empirical risk), and number of training points, such abstract framework is also addressed for inverse problems [111]. 根据这段内容生成markdown表格放在md代码框中，不要考虑论文编号，只考虑什么人针对什么PDE干了什么

Qwen3-Max

Md
编辑
1
|
作者
|
PDE 类型
|
工作内容
|
2
|
|
|
|
|
3

2.4.3罗列作者表格

注解：

[1]前向和倒向问题是什么？ 知识来源：deepseek

介绍一下偏微分方程中的前向和倒向问题，我需要将内容复制到思源笔记中，正式内容请放在markdown代码框中，注意公式或者特殊符号应该用美元符框住方便编译



当然可以。以下是关于偏微分方程中前向（正问题）和倒向（反问题）问题的介绍，内容已按您的要求整理在 Markdown 代码框中，并使用了美元符号标注公式。

markdown

复制 下载

偏微分方程中的前向问题与倒向问题

在偏微分方程的理论与应用中，根据已知信息和待求解信息的不同，问题通常被划分为两大类：****前向问题****和****倒向问题****。这两类问题在数学性质、求解难度和应用场景上有着根本性的区别。

1. 前向问题



给 DeepSeek 发送消息

深度思考

联网搜索

